

Unit 2: Problem Solving and Search Techniques

Problem Solving in Artificial Intelligence

Problem-solving in Artificial Intelligence (AI) is the process of identifying a goal, analyzing available options, and selecting the most effective sequence of actions to achieve the desired outcome. It enables intelligent systems to reason, search for solutions, and make decisions in complex situations.

- Enables AI agents to make goal-oriented decisions in different environments.
- Forms the foundation of applications such as robotics, planning, navigation, and game playing.

Problem Formulation in AI

Problem formulation is the process of defining a problem in a structured way so that an AI system can identify and achieve its goal efficiently.

- **Initial State:** Represents the starting point from which the AI agent begins solving the problem.
- **Actions:** Defines the possible operations or choices available to the agent at a given state.
- **Transition Model:** Describes how the state of the system changes after an action is performed.
- **Goal Test:** Determines whether the current state satisfies the desired goal.
- **Cost Function:** Measures the cost of reaching the goal and helps identify the most efficient solution path.

Working

- **Goal Formulation:** The AI identifies the objective it needs to achieve. A clearly defined goal helps the agent focus on the desired outcome and guides the problem-solving process.

- **Problem Definition:** The problem is clearly specified by identifying the initial state, goal state, constraints, and available actions. This establishes the framework for finding a solution.
- **Defining the State Space:** The AI represents all possible states and transitions that can occur while solving the problem. This helps the agent explore different paths toward the goal.
- **Testing the Goal State:** The system checks whether the current state satisfies the desired goal conditions. If the goal is reached, the search process can terminate.
- **Calculating Cost of Path:** The AI evaluates the cost associated with different solution paths, such as time, distance, or resources, and selects the most efficient path.

Techniques

Several techniques are available to solve different types of problems in AI. The choice of technique depends on the nature of the problem and the resources available.

1. Search Algorithms

Search Algorithms are important for exploring possible solutions. There are two main categories:

- **Uninformed Search:** These algorithms such as breadth-first and depth-first search, don't use additional knowledge about the problem and explore all possible paths.
- **Informed Search:** Algorithms like A* use heuristics or domain-specific knowledge to guide the search toward more promising paths making the search process more efficient.

2. Constraint Satisfaction Problems (CSP)

In CSPs the goal is to find solutions that satisfy a set of constraints. Techniques like backtracking, constraint propagation and local search are used to explore the solution space efficiently while ensuring that all constraints are met.

3. Optimization Techniques

AI often addresses optimization problems where the goal is to find the best possible solution among many options. Techniques include:

1. Linear programming for solving problems involving linear relationships.
2. Dynamic programming for breaking down complex problems into smaller subproblems.
3. Evolutionary algorithms like genetic algorithms that simulate natural selection to find optimal solutions.

4. Machine Learning

Machine learning allows AI systems to learn from data and improve over time. The three major learning types are:

1. Supervised Learning: Learning from labeled data to make predictions or classifications.
2. Unsupervised Learning: Discovering patterns in data without labeled outputs.
3. Reinforcement Learning: Learning through trial and error, receiving feedback in the form of rewards or penalties.

5. Natural Language Processing (NLP)

Natural Language Processing (NLP) helps AI systems understand and generate human language. Techniques like tokenization, Part-of-speech (PoS) and named entity recognition are important for chatbots, language translation and text summarization.

Applications

- **Robotics and Navigation:** Enables robots to navigate environments and perform tasks autonomously.
- **Game Playing:** Helps AI analyze strategies and make optimal decisions in games.
- **Medical Diagnosis:** Assists in identifying diseases by analyzing medical data and symptoms.
- **Route Planning and Logistics:** Finds efficient routes to reduce travel time and operational costs.
- **Recommendation Systems:** Suggests relevant products, services, or content based on user preferences.

Advantages

- **Efficiency:** Solves problems quickly by processing large amounts of data.
- **Accuracy:** Produces reliable results when trained with quality data.

- **Automation:** Reduces human effort by handling repetitive tasks automatically.
- **Adaptability:** Learns from new data and improves performance over time.
- **Cost-Effectiveness:** Increases productivity and reduces operational costs.

Challenges

- **Complexity:** Some problems require significant computational resources.
- **Data Quality:** Poor or biased data can affect decision accuracy.
- **Interpretability:** Advanced AI models can be difficult to understand and explain.
- **Ethics and Bias:** AI systems may inherit biases present in training data.

State space representation in AI

State space representation in AI models a problem as a collection of all possible configurations, or **states**, connected by legal **actions**. It defines an initial starting point and a target goal, letting search algorithms navigate the environment like a graph to find a solution.

Core Components

- **Initial State:** The starting configuration or position where the problem begins.
- **State Space:** The exhaustive set of all possible configurations reachable from the initial state.
- **Operators (Actions):** Valid rules or moves that transition the system from one state to another.
- **Goal State:** The target condition that signifies the problem has been successfully solved.
- **Path Cost:** A numerical value assigned to a path, reflecting the cost or number of steps taken.

How It Works

- The problem is mapped as a graph or a tree where **nodes are states** and **edges are actions/transitions**.
- Search algorithms like Breadth-First Search (BFS), Depth-First Search (DFS), or A* systematically traverse these nodes to discover a valid path from the start to the goal.

Uninformed Search Algorithms in AI

Uninformed Search Algorithms, also called Blind Search Algorithms, are search methods that find a solution using only the problem's initial state, available actions and goal state, without any additional guidance or heuristic information.

- Use only the initial state, available actions and goal state to find a solution.
- Do not rely on heuristic information or prior knowledge.
- Explore the search space in a systematic manner.
- Commonly applied in pathfinding, puzzle-solving and state-space search problems.
- Can be less efficient because many unnecessary states may be explored before reaching the goal.

Types of Uninformed Search Algorithms

1. Breadth-First Search (BFS)

Breadth First Search (BFS) is a graph traversal algorithm that starts from a source node and explores the graph level by level. First, it visits all nodes directly adjacent to the source. Then, it moves on to visit the adjacent nodes of those nodes, and this process continues until all reachable nodes are visited.

- BFS is different from DFS in a way that closest vertices are visited before others. We mainly traverse vertices level by level.
- Popular graph algorithms like Dijkstra's shortest path, Kahn's Algorithm, and Prim's algorithm are based on BFS.
- BFS itself can be used to detect cycles in a directed and undirected graph, find shortest path in an unweighted graph and many more problems.

2. Depth-First Search (DFS)

In Depth First Search (or DFS) for a graph, we traverse all adjacent vertices one by one. When we traverse an adjacent vertex, we completely finish the traversal of all vertices reachable through that adjacent vertex. This is similar to Preorder Traversal of Binary Tree, where we first completely traverse the left subtree and then move to the right subtree. The key difference is that, unlike trees, graphs may contain cycles (a node may be visited more than once). To avoid processing a node multiple times, we use a boolean visited array.

3. Uniform-Cost Search (UCS)

Uniform Cost Search (UCS) is an uninformed search algorithm used to find the lowest-cost path between a start node and a goal node in a weighted graph. It systematically explores paths based on their cumulative cost, making it suitable for cost-sensitive pathfinding problems.

- Considers the total path cost rather than the number of steps taken.
- Guarantees the optimal solution when all edge costs are non-negative.
- Uses a priority queue to efficiently select the next node with the lowest cost.

Key Concepts

- **Priority Queue:** UCS uses a priority queue to store nodes and always selects the node with the lowest total cost for exploration.
- **Path Cost:** Calculates the cumulative cost from the start node to the current node and prioritizes nodes with lower costs.
- **Node Expansion:** Explores nodes in order of increasing path cost, giving preference to cheaper paths first.
- **Optimal Solution:** Stops when the goal node is reached through the lowest-cost path, ensuring the best solution when all edge costs are non-negative.

Working

Uniform Cost Search (UCS) operates by always exploring the path with the lowest cumulative cost first. It uses a priority queue to keep track of nodes and their path costs.

1. **Initialization:** The algorithm begins at the starting node and assigns it a cost of 0 because no movement has been made yet.
2. **Node Expansion:** The node with the lowest path cost is selected from the priority queue, and its neighboring nodes are explored.
3. **Exploring Neighbors:** For each neighboring node, UCS calculates the total cost to reach it from the start node. The node is then added to the priority queue, or its cost is updated if a cheaper path is found.
4. **Goal Check:** After expanding a node, the algorithm checks if it has reached the goal node. If the goal is reached, the algorithm returns the total cost to reach this node and the path taken.

5. **Repetition:** This process repeats until the priority queue is empty or the goal is reached.

Informed Search Algorithms in Artificial Intelligence

Informed search algorithms in AI are search methods that use extra knowledge, called heuristics, to prioritize which paths to explore. By estimating how close each possible step is to the goal, these algorithms can find solutions more quickly and efficiently than uninformed or “blind,” search. They are widely used in AI for tasks like pathfinding and puzzle solving because they help navigate large, complex search spaces.

Greedy Best first search algorithm

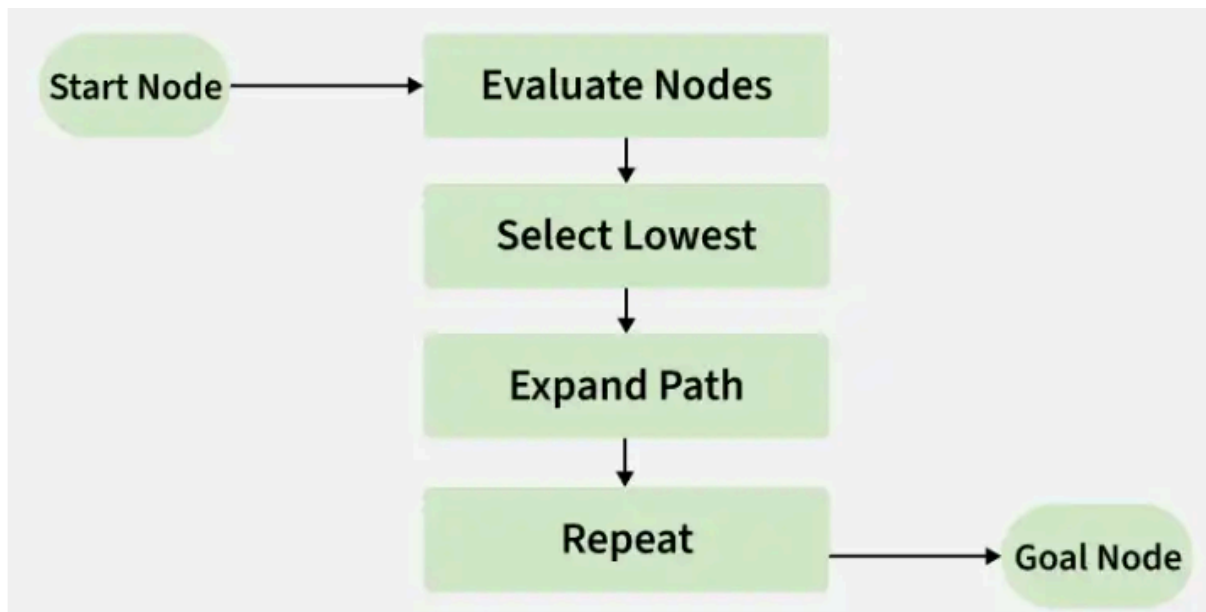
Greedy Best-First Search is an AI search algorithm used to find a path from a starting node to a goal node by selecting the path that appears most promising at each step. It uses heuristic values to guide the search toward the goal efficiently.

- Prioritizes faster exploration over guaranteed optimal solutions
- Expands the node with the lowest heuristic value
- Reduces search effort by focusing on promising paths
- Commonly applied in navigation systems and pathfinding problems

Working

Greedy Best-First Search works by selecting the node that appears closest to the goal using heuristic values and continues expanding the most promising path until the destination is reached.

- Starts from the initial node and evaluates neighboring nodes using heuristic values
- Selects the node with the lowest heuristic value for expansion
- Expands the most promising path instead of exploring all possible paths
- Repeats the process until the goal node is reached
- Uses heuristic estimates to reduce search time and guide traversal efficiently



2. A* Search

A* Search finds the shortest path by combining two things: how far we have already gone and a guess of how far is left to go. It checks both actual progress and potential, so it's very reliable if our guess (heuristic) is good. It uses a combined cost function

$$f(n)=g(n)+h(n)$$

$f(n)=g(n)+h(n)$ actual path cost so far

$g(n)$

$g(n)$ plus heuristic estimate

$h(n)$

$h(n)$ to the goal. Finds both shortest and most cost-effective path with an admissible heuristic.

- **Advantage:** Complete and optimal (when using an admissible heuristic); widely applicable.
- **Limitation:** Can consume high memory for large graphs or spaces

Hill Climbing in Artificial Intelligence

Last Updated :25 Apr, 2026



Hill climbing is a heuristic search algorithm that belongs to the family of local search methods. It is designed to solve problems where the goal is to find an optimal (or near-optimal) solution by iteratively moving from the current state to a better neighboring state, according to a heuristic or evaluation function.

- It is an optimisation technique used in artificial intelligence (AI) to find solutions for a wide variety of problems.
- It operates on the principle of incrementally improving a solution by making local changes and evaluating their merit.
- Its simplicity, intuitive logic and adaptability to different problems make it a go-to method.

Hill Climbing Algorithms

Hill climbing follows these steps:

1. **Initial State:** Start with an arbitrary or random solution (initial state).
2. **Neighboring States:** Identify neighboring states of the current solution by making small adjustments (mutations or tweaks).
3. **Move to Neighbor:** If one of the neighboring states offers a better solution (according to some evaluation function), move to this new state.
4. **Termination:** Repeat this process until no neighboring state is better than the current one. At this point, we have reached a local maximum or minimum.

Features of Hill Climbing Algorithm

1. Variant of Generating and Testing Algorithm: Hill Climbing is a specific variant of the generating and testing algorithms. The process involves: This iterative feedback mechanism allows Hill Climbing to refine its search by using information from previous evaluations to inform future moves in the search space.

- **Generating possible solutions:** The algorithm creates potential solutions within the search space.
- **Testing solutions:** Each generated solution is evaluated to determine if it meets the desired criteria.
- **Iteration:** If a satisfactory solution is found, the algorithm terminates; otherwise, it returns to the generation step.

2. Greedy Approach: Hill Climbing algorithm uses greedy approach, meaning that at each step, it moves in the direction that optimizes the objective function. This strategy aims to find the optimal solution efficiently by making the best immediate choice without considering the overall problem context.

Types of Hill Climbing in Artificial Intelligence

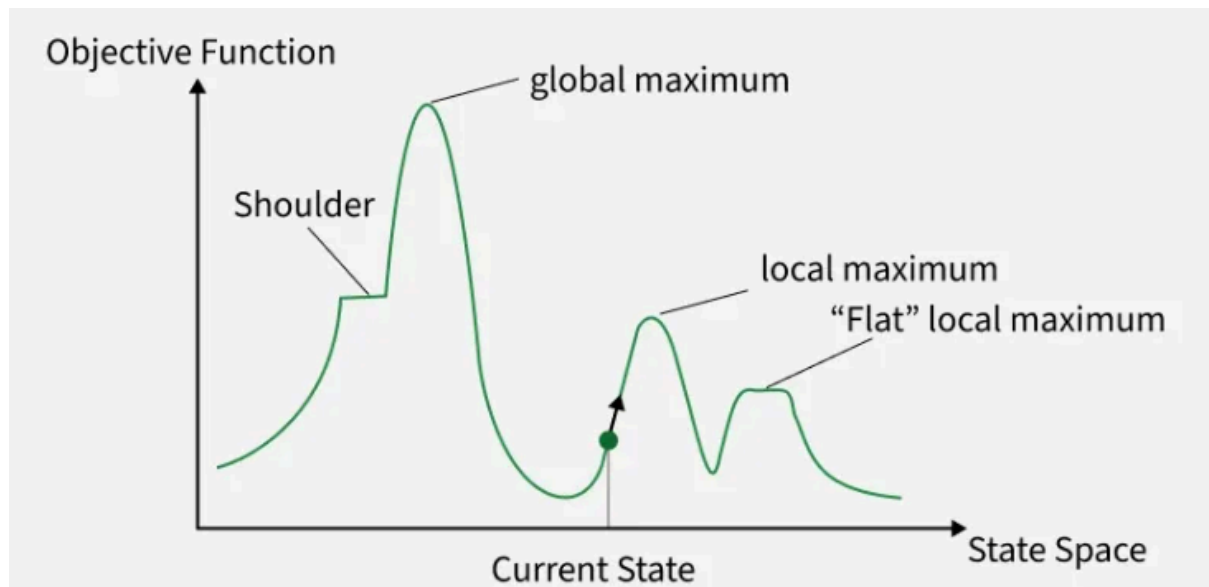
1. Simple Hill Climbing Algorithm: Simple Hill Climbing is a straightforward variant of hill climbing where the algorithm evaluates each neighbouring node one by one and selects the first node that offers an improvement over the current one.

2. Steepest-Ascent Hill Climbing: Steepest-Ascent Hill Climbing is an enhanced version of simple hill climbing. Instead of moving to the first neighboring node that improves the state, it evaluates all neighbors and moves to the one offering the highest improvement (steepest ascent).

3. Stochastic Hill Climbing: Stochastic Hill Climbing introduces randomness into the search process. Instead of evaluating all neighbors or selecting the first improvement, it selects a random neighboring node and decides whether to move based on its improvement over the current state.

State-Space Diagram in Hill Climbing

The state-space diagram is a visual representation of all possible states the search algorithm can reach, plotted against the values of the objective function (the function we aim to maximize).



Sta-Space Diagram

In the state-space diagram:

- **X-axis:** Represents the state space which includes all the possible states or configurations that the algorithm can reach.
- **Y-axis:** Represents the values of the objective function corresponding to each state.

The optimal solution in the state-space diagram is represented by the state where the **objective function** reaches its maximum value, also known as the **global maximum**.

Key Regions in the State-Space Diagram

1. **Local Maximum:** A local maximum is a state better than its neighbors but not the best overall. While its objective function value is higher than nearby states, a global maximum may still exist.
2. **Global Maximum:** The global maximum is the best state in the state-space diagram where the objective function achieves its highest value. This is the optimal solution the algorithm seeks.
3. **Plateau/Flat Local Maximum:** A plateau is a flat region where neighboring states have the same objective function value, making it difficult for the algorithm to decide on the best direction to move.

4. **Ridge:** A ridge is a higher region with a slope which can look like a peak. This may cause the algorithm to stop prematurely, missing better solutions nearby.
5. **Current State:** The current state refers to the algorithm's position in the state-space diagram during its search for the optimal solution.
6. **Shoulder:** A shoulder is a plateau with an uphill edge allowing the algorithm to move toward better solutions if it continues searching beyond the plateau.

Advantages of Hill Climbing Algorithm

1. **Simplicity and Ease of Implementation:** Hill Climbing is a simple and intuitive algorithm that is easy to understand and implement making it accessible for developers and researchers.
2. **Versatility:** The algorithm can be applied to a wide variety of optimization problems, including those with large search spaces and complex constraints.
3. **Efficiency in Finding Local Optima:** Hill Climbing is often highly efficient at finding local optima making it a suitable choice for problems where a good solution is required quickly.
4. **Customizability:** The algorithm can be easily modified or extended to incorporate additional heuristics or constraints allowing for more tailored optimization approaches.

Applications of Hill Climbing in AI

- **Pathfinding:** It is used in AI systems that need to navigate or find the shortest path between points such as in robotics or game development.
- **Optimization:** It can be used for solving optimization problems where the goal is to maximize or minimize a particular objective function such as scheduling or resource allocation problems.
- **Game AI:** In certain games, AI uses hill climbing to evaluate and improve its position relative to an opponent's.
- **Machine Learning:** It is sometimes used for hyperparameter tuning where the algorithm iterates over different sets of hyperparameters to find the best configuration for a machine learning model.

Constraint Satisfaction Problems (CSP)

A **Constraint Satisfaction Problem** is a mathematical problem where the solution must meet a number of constraints. In CSP the objective is to assign values to variables such that all the constraints are satisfied. Many AI applications use CSPs to solve decision-making problems that involve managing or arranging resources under strict guidelines. Common applications of CSPs include:

- **Scheduling:** It assigns resources like employees or equipment while respecting time and availability constraints.
- **Planning:** Organize tasks with specific deadlines or sequences.
- **Resource Allocation:** Distributing resources efficiently without overuse

CSPs are composed of three key elements:

1. Variables: These are the things we need to find values for. Each variable represents something that needs to be decided. For example, in a Sudoku puzzle each empty cell is a variable that needs a number. Variables can be of different types like yes/no choices (Boolean), whole numbers (integers) or categories like colors or names.

2. Domains: This is the set of possible values that a variable can have. The domain tells us what values we can choose for each variable. In Sudoku the domain for each cell is the numbers 1 to 9 because each cell must contain one of these numbers. Some domains are small and limited while others can be very large or even infinite.

3. Constraints: These are the rules that restrict how variables can be assigned values. Constraints define which combinations of values are allowed. There are different types of constraints:

- **Unary constraints** apply to a single variable like "this cell cannot be 5".
- **Binary constraints** involve two variables like "these two cells cannot have the same number".
- **Higher-order constraints** involve three or more variables like "each row in Sudoku must have all numbers from 1 to 9 without repetition"

Types of Constraint Satisfaction Problems

CSPs can be classified into different types based on their constraints and problem characteristics:

1. **Binary CSPs:** In these problems each constraint involves only two variables. Like in a scheduling problem the constraint could specify that task A must be completed before task B.
2. **Non-Binary CSPs:** These problems have constraints that involve more than two variables. For instance in a seating arrangement problem a constraint could state that three people cannot sit next to each other.
3. **Hard and Soft Constraints:** Hard constraints must be strictly satisfied while soft constraints can be violated but at a certain cost. This is often used in real-world applications where not all constraints are equally important.

Representation of Constraint Satisfaction Problems (CSP)

In CSP it involves the interaction of variables, domains and constraints. Below is a structured representation of how CSP is formulated:

1. **Finite Set of Variables ($V_1, V_2, \dots, V_n$):** The problem consists of a set of variables each of which needs to be assigned a value that satisfies the given constraints.
2. **Non-Empty Domain for Each Variable ($D_1, D_2, \dots, D_n$):** Each variable has a domain, a set of possible values that it can take. For example, in a Sudoku puzzle the domain could be the numbers 1 to 9 for each cell.
3. **Finite Set of Constraints ($C_1, C_2, \dots, C_m$):** Constraints restrict the possible values that variables can take. Each constraint defines a rule or relationship between variables.
4. **Constraint Representation:** Each constraint C_i is represented as a pair of (scope, relation) where:
 - **Scope:** The set of variables involved in the constraint.
 - **Relation:** A list of valid combinations of variable values that satisfy the constraint.

Applications of CSPs in AI

CSPs are used in many fields because they are flexible and can solve real-world problems efficiently. Here are some common applications:

1. **Scheduling:** They help in planning things like employee shifts, flight schedules and university timetables. The goal is to assign

tasks while following rules like time limits, availability and priorities.

2. **Puzzle Solving:** Many logic puzzles such as Sudoku, crosswords and the N-Queens problem can be solved using CSPs. The constraints make sure that the puzzle rules are followed.
3. **Configuration Problems:** They help in selecting the right components for a product or system. For example when building a computer it ensure that all selected parts are compatible with each other.
4. **Robotics and Planning:** Robots use CSPs to plan their movements, avoid obstacles and complete task efficiently. For example a robot navigating a warehouse must avoid crashes and minimize energy use.
5. **Natural Language Processing (NLP):** In NLP they help with tasks like breaking sentences into correct grammatical structures based on language rules.

Benefits of CSPs in AI

1. **Standardized Representation:** They provide a clear and structured way to define problems using variables, possible values and rules.
2. **Efficiency:** Smart search techniques like backtracking and forward-checking help to reduce the time needed to find solutions.
3. **Flexibility:** The same CSP methods can be used in different areas without needing expert knowledge in each field.

Challenges in Solving CSPs

1. **Scalability:** When there are too many variables and rules the problem becomes very complex and finding a solution can take too long.
2. **Changing Problems (Dynamic CSPs):** In real life conditions and constraints can change over time requiring the CSP solution to be updated.
3. **Impossible or Overly Strict Problems:** Sometimes a CSP may not have a solution because the rules are too strict. In such cases adjustments or compromises may be needed to find an acceptable solution.

Game Playing in Artificial Intelligence

Game Playing is an important domain of artificial intelligence. Games don't require much knowledge; the only knowledge we need to provide is the rules, legal moves and the conditions of winning or losing the game. Both players try to win the game. So, both of them try to make the best move possible at each turn.

- Searching techniques like BFS(Breadth First Search) are not efficient for this as the branching factor is very high, so searching will take a lot of time.
- Game playing in AI is an active area of research and has many practical applications, including game development, education, and military training.
- By simulating game playing scenarios, AI algorithms can be used to develop more effective decision-making systems for real-world applications.
- The most common search technique in game playing is the **Minimax search procedure**. It is depth-first depth-limited search procedure. It is used for games like chess and tic-tac-toe.

The Minimax Search Algorithm

One of the most common search techniques in game playing is the **Minimax algorithm**, which is a depth-first, depth-limited search procedure. Minimax is commonly used for games like chess and tic-tac-toe.

Key Functions in Minimax:

1. **MOVEGEN**: Generates all possible moves from the current position.
2. **STATICEVALUATION**: Returns a value based on the quality of a game state from the perspective of two players.

In a two-player game, one player is referred to as PLAYER1 and the other as PLAYER2. The Minimax algorithm operates by backing up values from child nodes to their parent nodes. PLAYER1 tries to maximize the value of its moves, while PLAYER2 tries to minimize the value of its moves. The algorithm recursively performs this procedure at each level of the game tree.

Example of Minimax: The game starts with PLAYER1. The algorithm generates four levels of the game tree. The values for nodes H, I, J, K, L, M, N, and O are provided by the STATICEVALUATION function. Level 3 is a maximizing level, so each node at this level takes the maximum value of its children. Level 2 is a minimizing level, where each node takes the minimum value of its children. After this process, the value of node A is calculated as 23, meaning that PLAYER1 should choose move C to maximize the chances of winning.

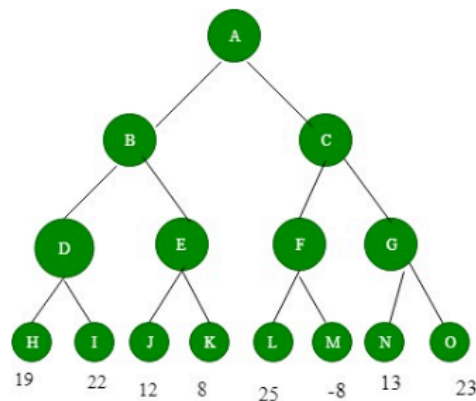


Figure 1: Before backing-up of values

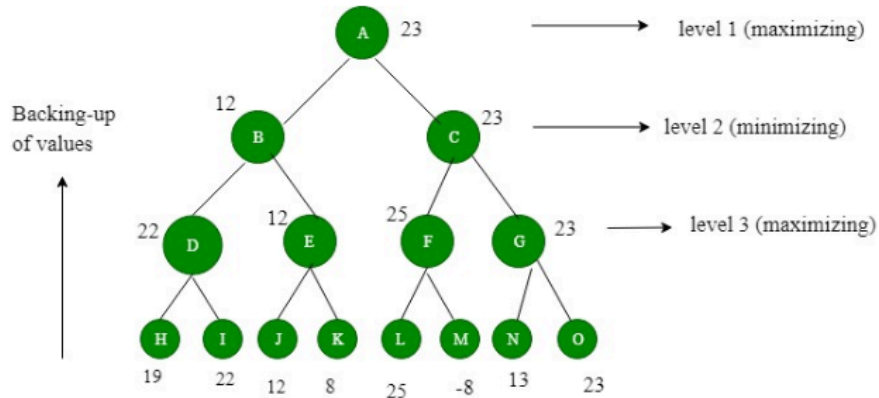
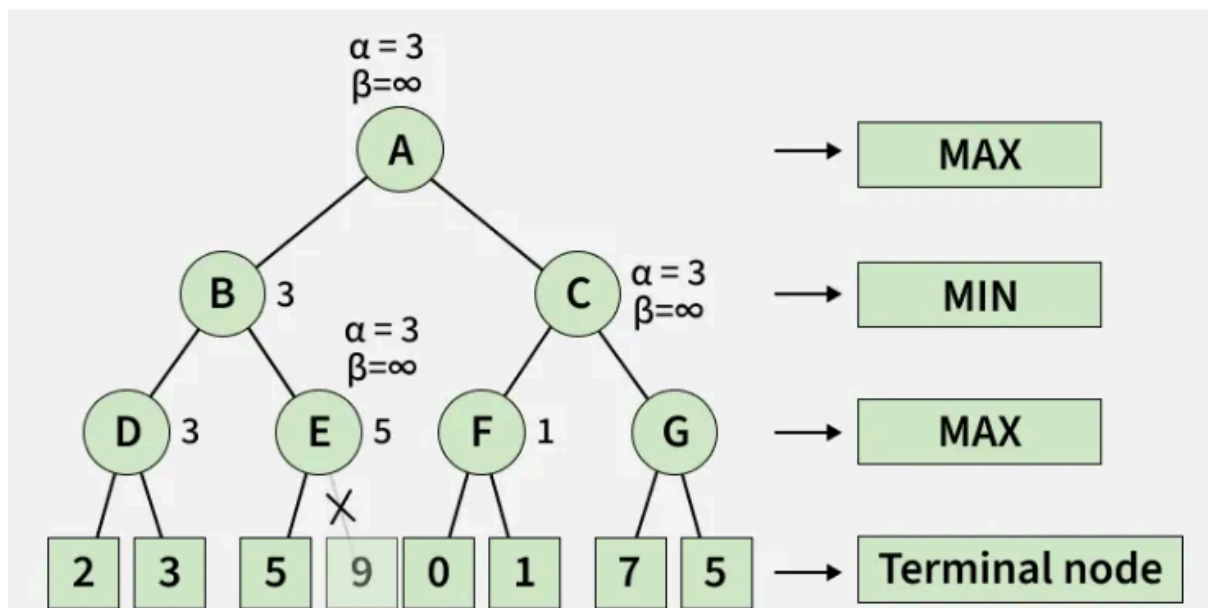


Figure 2: After backing-up of values We assume that PLAYER1 will start the game.

Alpha-Beta Pruning

Alpha-Beta Pruning is an optimisation technique that significantly improves the efficiency of Minimax. It removes (prunes) branches that cannot possibly affect the final decision, avoiding unnecessary computations.



The image shows a Minimax game tree with Alpha-Beta pruning, where MAX and MIN levels evaluate child nodes to compute the optimal value. Branches that cannot affect the final outcome like the one under node E are pruned, reducing unnecessary exploration.

Pruning is the process of cutting off branches in the search tree that cannot influence the final outcome, saving time and computation.

During Minimax traversal, two values are maintained:

- **Alpha (α):** The best score the Maximizer can guarantee so far.
- **Beta (β):** The best score the Minimizer can guarantee so far.
- **Pruning Condition:** If $\alpha \geq \beta$ the branch is cut off because it cannot affect the final decision.

How Alpha-Beta Pruning Works

Step 1: Initialization

Set initial values:

$$\alpha = -\infty \quad \beta = +\infty$$

These represent the worst possible scores for Maximizer and Minimizer

Step 2: Max Node Evaluation

For each child of a Max node recursively evaluate using Minimax with Alpha-Beta pruning. Update alpha:

$$\alpha = \max(\alpha, \text{value})$$

If $\alpha \geq \beta$ stop evaluating remaining children (β -cutoff)

Step 3: Min Node Evaluation

For each child of a Min node Recursively evaluate using Minimax with Alpha-Beta pruning Update Beta:

$$\beta = \min(\beta, \text{value}) \text{ If } \beta \leq \alpha \text{ stop evaluating remaining children } (\alpha\text{-cutoff})$$

Applications

Alpha-Beta Pruning is widely used in adversarial game playing AI including:

- **Chess:** Enables engines to explore more moves deeply and efficiently.
- **Checkers:** Enhances search performance, making AI harder to defeat.
- **Othello/Reversi:** Improves strategy evaluation during gameplay.
- **Tic-Tac-Toe:** Ensures optimal play with minimal computation.

Advantages

- **Improves Efficiency:** Greatly reduces the number of nodes evaluated compared to plain Minimax, allowing deeper searches in the same time.
- **Maintains Optimality:** Pruning does not change the final result; the chosen move is still the optimal one.
- **Faster Decision Making:** Helps game agents respond quickly, especially in time-restricted games.
- **Better Resource Usage:** Saves computation and memory by avoiding unnecessary branches.

Limitations

- **Still Expensive for Large Trees:** Although improved, the algorithm remains slow for extremely deep or high-branching games.
- **Depends on Move Ordering:** Works best when good moves are evaluated first; poor ordering reduces pruning efficiency.

- **Requires Heuristics:** Effectiveness often depends on strong evaluation functions or domain knowledge.
- **Hard to Optimize for Complex Games:** Designing good heuristics and ordering strategies can be challenging.